

Server-Factory-Core-Framework

El motor compartido detrás de cada Server Factory.

Resumen

Core Framework es el marco de trabajo Kotlin que sustenta la familia de herramientas de aprovisionamiento Server Factory. Proporciona el motor común y las abstracciones sobre las que se construyen proyectos como Mail Server Factory, de modo que cada “fábrica” reutiliza un mismo fundamento probado en batalla en lugar de volver a implementar primitivas de aprovisionamiento.

Descripción breve

El marco de trabajo Kotlin compartido en la base del ecosistema Server Factory. Suministra el motor de aprovisionamiento común, las abstracciones de conexión y la maquinaria de pasos de instalación que consumen las fábricas descendentes (Mail Server Factory, Web Service Factory, SonarQube Factory y otras).

Descripción detallada

Core Framework es esa pieza silenciosa de ingeniería que hace posible toda la familia Server-Factory: el motor reutilizable sobre el que se construye cada producto “fábrica” individual (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory). El enfoque de Server Factory es declarativo: el usuario describe la infraestructura que desea mediante una configuración, y una fábrica interpreta esa descripción para instalar e inicializar el software en un sistema destino. Core Framework es donde reside la maquinaria común a este patrón: las abstracciones de conexión y transporte que alcanzan todo tipo de destinos, el modelo de pasos de instalación que codifica *cómo* se aprovisiona el software, y la infraestructura compartida que, de otro modo, cada fábrica tendría que implementar por su cuenta. Es la respuesta a una pregunta estructural que toda cadena de

herramientas multiproducto termina enfrentando: *¿dónde va el motor compartido?* Y acertar en esa respuesta una sola vez es lo que mantiene cohesionada a la familia, en lugar de fragmentarla en cuatro proveedores sutilmente distintos. Al centralizar esto en un único marco de trabajo Kotlin, la familia evita duplicar la lógica de aprovisionamiento en los productos y garantiza un comportamiento coherente: un tipo de conexión o una primitiva de instalación mejorada en Core Framework beneficia a todas las fábricas descendentes. Está compuesto casi en su totalidad por Kotlin (aproximadamente 990K bytes de Kotlin con una fina capa de Shell), lo que refleja su papel como biblioteca de código en lugar de una colección de scripts. Los repositorios descendentes lo referencian como su dependencia canónica (Parallels-Utils, Qemu-Utils, Utils y los paquetes de Definiciones aluden al repositorio de Core Framework como el eje del ecosistema). Su archivo README es intencionalmente minimalista —es infraestructura para otros proyectos, versionada mediante `version.txt/version_code.txt`— y precede al trabajo posterior de AI, lo que lo convierte en parte del legado maduro de herramientas DevOps de la organización.

Por qué lo creamos

Cada herramienta de aprovisionamiento necesita el mismo núcleo: formas de conectarse a los destinos y pasos para instalar/configurar el software. Reconstruirlo para cada producto fragmentaría el comportamiento y multiplicaría los errores. Core Framework lo centraliza para que todas las fábricas compartan un mismo motor confiable.

Por qué es un cambio de juego

Es el punto de mayor apalancamiento en toda la familia: un tipo de conexión reforzado o una primitiva de instalación mejorada aquí propaga esa corrección y capacidad a todas las fábricas de una vez, de modo que toda la cadena de herramientas se beneficia de una sola inversión. Es la filosofía de *“construir una vez, reutilizar en todas partes”* aplicada donde más rinde: la capa fundacional de la automatización de infraestructura, donde una corrección en el lugar adecuado lo soluciona todo en los niveles inferiores.

Qué es innovador

- Un único marco de aprovisionamiento reutilizable que abstrae la lógica de conexión y los pasos de instalación.
- Clara separación entre el motor (Marco Central) y las fábricas específicas de cada producto.

- Distribución con versiones fijadas (version.txt/
version_code.txt) para un consumo reproducible.

Desafíos y soluciones

- **Evitar la duplicación de la lógica de aprovisionamiento:** resuelto al extraer la maquinaria compartida en un único marco consumido por todas las fábricas.
- **Comportamiento consistente en todos los productos:** resuelto con abstracciones comunes para que los tipos de conexión y los pasos se comporten de manera idéntica en todas partes.
- **(NO VERIFICADO):** las API internas específicas no están documentadas en el archivo README público; considerar los detalles de la interfaz como no verificados más allá de “marco compartido consumido por las fábricas”.

Tecnologías empleadas (por qué y cómo)

- **Kotlin** — el marco completo (~990K bytes); el lenguaje de la familia Server Factory.
- **Shell** — scripts de soporte mínimos.
- **Gradle** — cadena de herramientas de compilación (coherente con el uso de ./gradlew de la familia).

Nota: GitHub marca el repositorio como una bifurcación dentro de la organización Server-Factory. No está centrado en AI; se presenta como la columna vertebral de la cadena de herramientas de aprovisionamiento.